

A Fast Parallel Median Filtering Algorithm Using Hierarchical Tiling

[SIGGRAPH Conference Papers '25: Proceedings of the Special Interest Group on Computer Graphics and Interactive Techniques Conference Conference Papers](#)

Louis Sugy, NVIDIA et al.; July 2025

Implementation & proposed improvements

by N. Ravi, VIT-AP Univ.

What is a median filter?

Median filtering is a **non linear, image filtering technique** that is very often used to filter out outlier values (noise) from a signal or image.

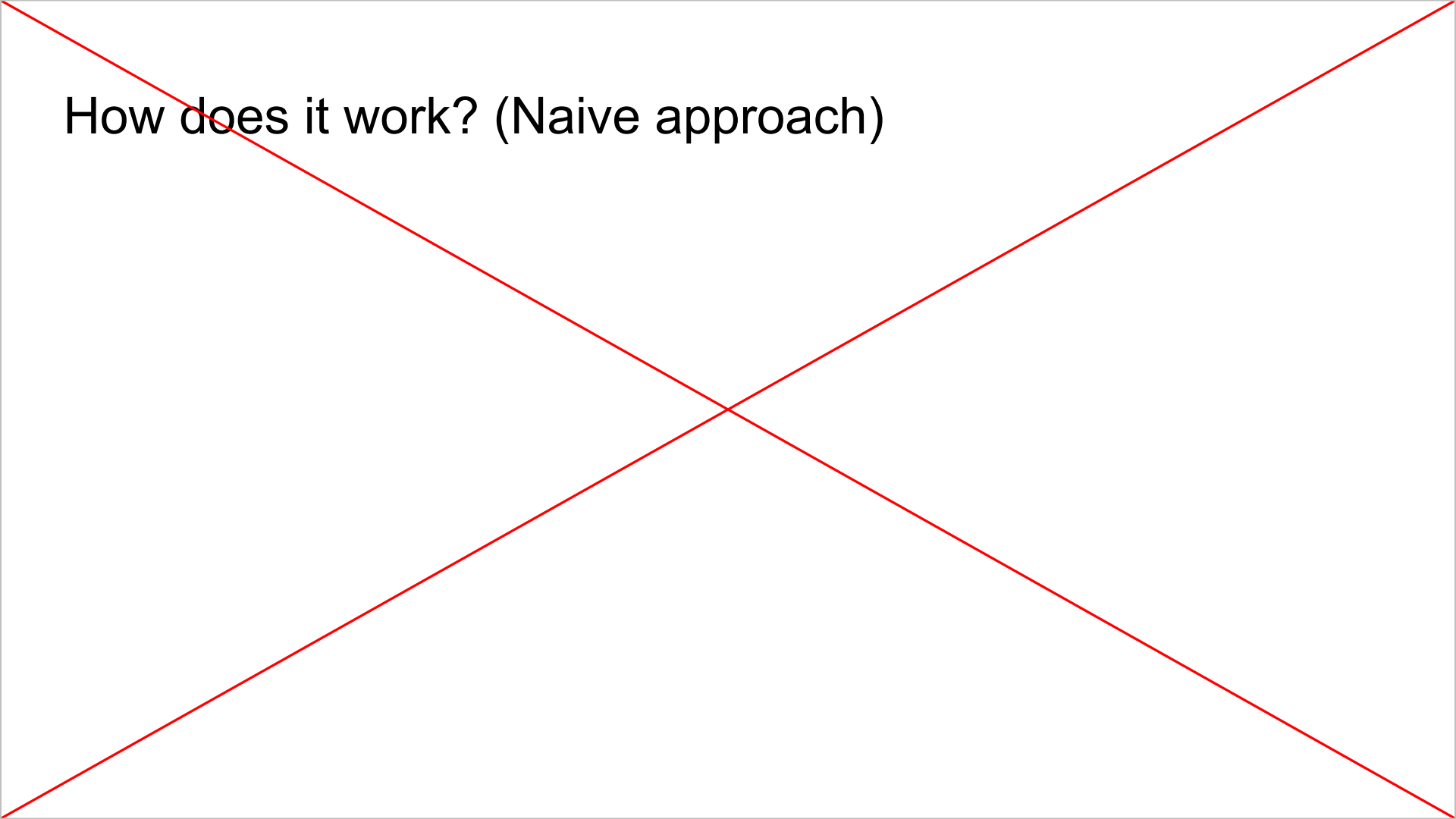
- Particularly effective at **removing “salt and pepper”** noise.
- Replaces each pixel with the **median** of neighbouring pixel values.
- Preserves edges better than a mean filter.
- Commonly used in pre-processing steps for image analysis/CV.

The size of the neighbourhood used (eg. 3x3, 17x17) determines the **smoothing strength**.

How does it work? (Naive approach)

- The algorithm is simple in idea; we replace the pixel with the median of the surrounding neighbours, to **discard the “effect” of pixels of outlier values** (close to 0 or 255 in the case of uint8)
- For each pixel, a **K x K window of neighbouring pixels** is taken.
- The pixel values in the window are **sorted**.
- **The median value replaces the center pixel.**
- This process is repeated for EVERY pixel in the image.

How does it work? (Naive approach)

A large red 'X' is drawn across the entire slide, from corner to corner, indicating that the content is incorrect or invalid.

Results of Median Filtering

- Without median filtering
- Signal noise near ears and whiskers



Results of Median Filtering

- With median filtering
- Signal noise smoothed out



Pitfalls with the naive approach

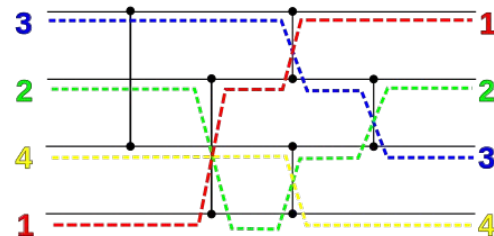
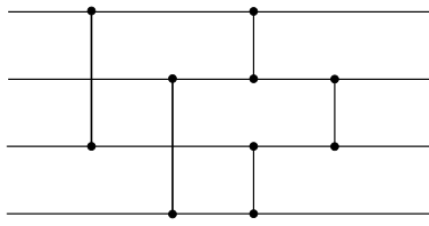
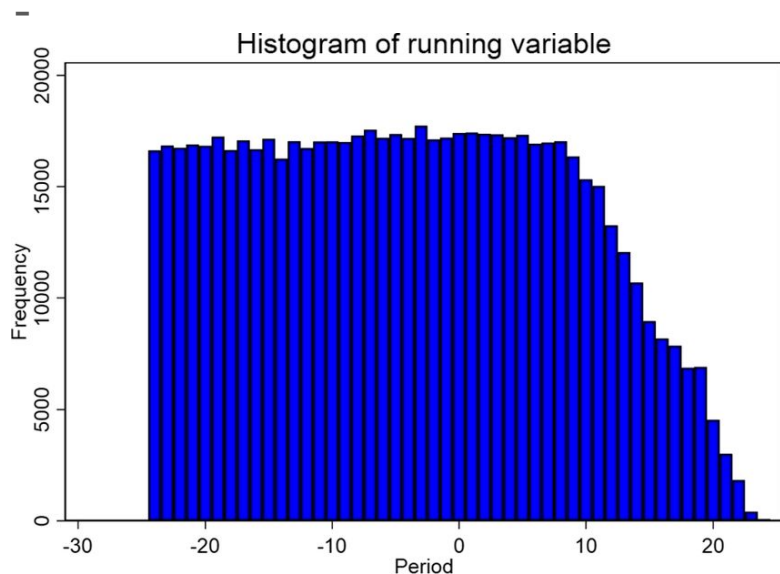
The **asymptotic complexity**, combined with the **non linear nature** of the algorithm leads to unreasonable runtime.

- Amortized complexity: **$O(W \times H \times K^2 \times \log(K^2))$**
- This leads to a runtime of **~2300 seconds for a 4000*4000 uint8 image**, on a modern, discrete consumer GPU!
- As the algorithm is non linear, we **cannot decompose** it into smaller, independent 1D operations, unlike FFT, Gaussian/mean filters.
- Hence, the naive approach is not acceptable for large kernels/images.

How do major libraries remedy this?

Major libraries like OpenCV or others, rely on either:

- A running histogram, to avoid sorting all values every time.
- Or a sorting network, for a $O(1)$ sort for small number of values.



Why are they still inefficient?

Histogram-based approaches:

- **Require maintaining and updating histograms for each window**, so you need to keep a track of how many pixels correspond to each valid pixel value (0-255 for uint8, **upto 65535 or more**, for higher bit depth..)
- Easily eats up tons of shared memory if we go higher than float64.

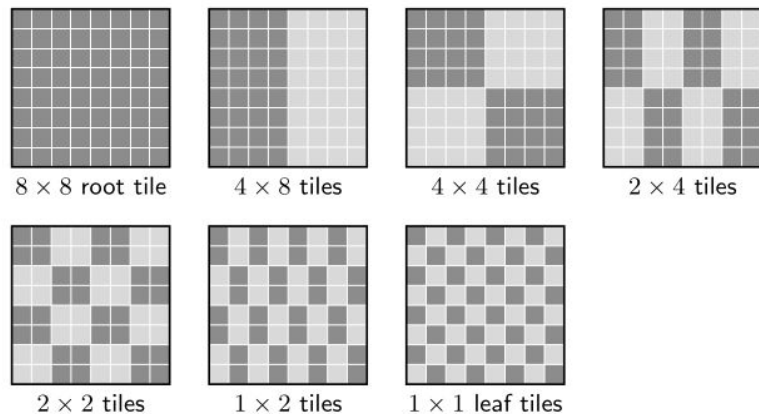
Sorting network approaches:

- **Fixed size and non adaptive.** Every comparison executed, no result caching.
- Can't draw a sorting network for every size of N.
- High register pressure and shared memory usage.

The Paper's Solution: **Hierarchical tiling**

- Proposes a faster median filtering algorithm, optimized for parallel processing.
- Uses **hierarchical tiling**; the **image is divided into smaller tiles**, then processed and merged hierarchically.
- This improves data locality, and reduces redundant computations.
- The results from the smaller computations are cached; algorithm is modified to be **data-aware**.
- **GPU-hardware friendly!** Registers are not pressured to keep the entire tile/image in memory.

The Paper's Solution: **Hierarchical tiling**



$$\begin{cases} t_w^{(i+1)} = \frac{t_w^{(i)}}{2}, & t_h^{(i+1)} = t_h^{(i)} & \text{if } t_w^{(i)} \geq t_h^{(i)} \\ t_w^{(i+1)} = t_w^{(i)}, & t_h^{(i+1)} = \frac{t_h^{(i)}}{2} & \text{else} \end{cases}$$

Fig. 4. Illustration of hierarchical tiling, starting from an 8×8 root tile. At each level, each tile is divided into two smaller tiles.

The Paper's Solution: Hierarchical tiling

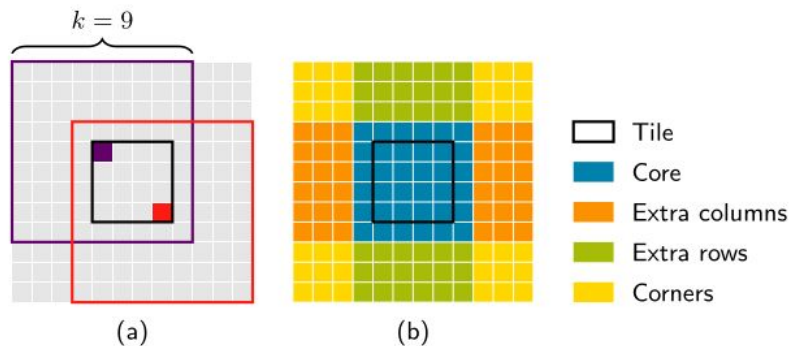


Fig. 5. Footprint of a 4×4 tile for a 9×9 kernel. (a) Outlines of the kernels for two pixels at opposite ends of the tile. (b) Partition of the tile's footprint into a core (the intersection of all 16 kernels), extra columns, extra rows, and corners.

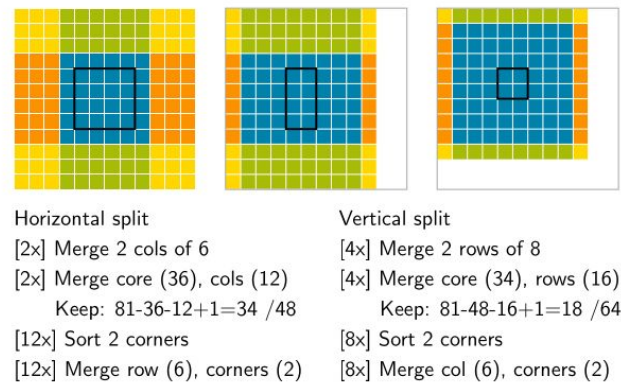


Fig. 6. First stages of the recursion for a 9×9 kernel using a 4×4 root tile, illustrated for the top-left child tile: the root tile is split horizontally into two 2×4 tiles, then again vertically into four 2×2 tiles.

Our Implementation/improvements

- Implements the paper faithfully.
- Focuses on **improving the performance** of the kernel.
- For the merger of children tiles, which the paper left up to the reader, we went with **Bitonic Odd-Even Sort**, as introduced in a different paper (*Kenneth, Batcher; 1968, "Sorting networks and their applications"*).
- The kernel is written in **CUDA/C++**.
- Observed an improvement of upto an average of **18x better speed, compared to OpenCV**, with the same kernel sizes.

(using cv::medianBlur() vs. our implementation)

Kernel Implementation Snippet

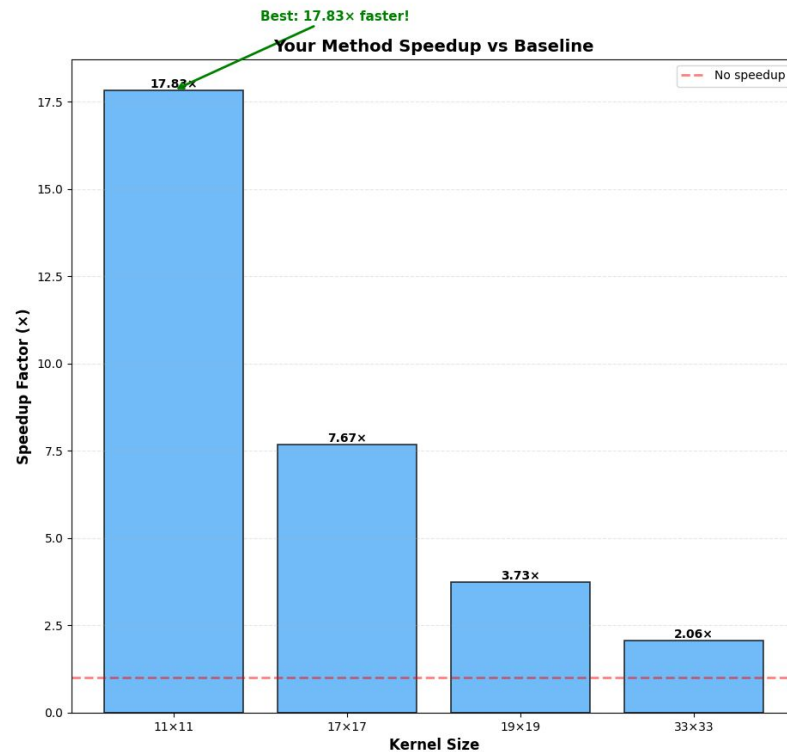
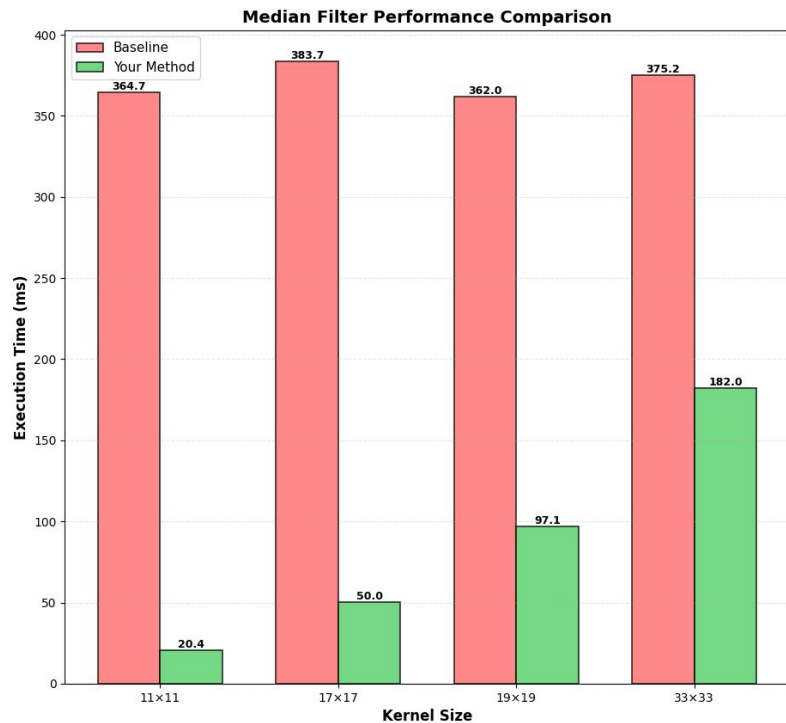
```
1 // @param T: template with pixel size (uchar)
2 // @param KSIZE: kernel size
3 // @param TILE_W: width split
4 // @param TILE_H: horizontal split
5 template <typename T, int KSIZE, int TILE_W, int TILE_H>
6 __global__ void __runKernel(const T* src, T* dst, int width, int height) {
7     const int x = blockIdx.x * blockDim.x + threadIdx.x;
8     const int y = blockIdx.y * blockDim.y + threadIdx.y;
9
10    if (x >= width || y >= height) return;
11
12    constexpr int WINDOW_SIZE = KSIZE * KSIZE;
13    T window[WINDOW_SIZE];
14    int radius = KSIZE / 2;
15    int count = 0;
16
17    for (int j = -radius; j <= radius; ++j) {
18        for (int i = -radius; i <= radius; ++i) {
19            int sx = x + i;
20            int sy = y + j;
21            int clamped_sx = max(0, min(width - 1, sx));
22            int clamped_sy = max(0, min(height - 1, sy));
23            window[count++] = src[clamped_sy * width + clamped_sx];
24        }
25    }
26
27    sort_network<T, WINDOW_SIZE>(window);
28
29    dst[y * width + x] = window[WINDOW_SIZE / 2];
30 }
31
```

Batcher's Odd-Even Bitonic Sort Implementation

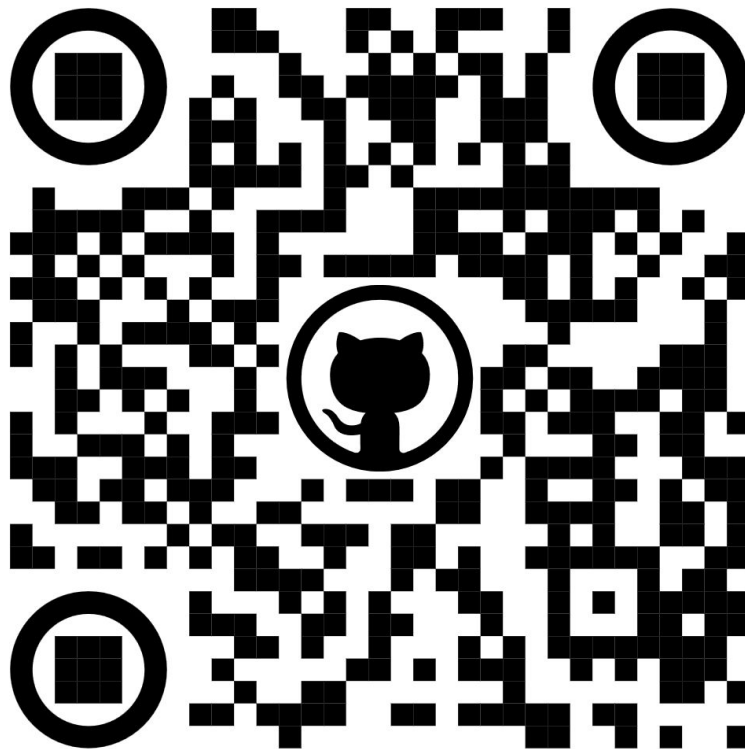


```
1 // bitonic odd even sort (batcher, 1968)
2 // CUDA code
3 template <typename T, int N>
4 __device__ void sort_network(T arr[N]) {
5     for (int i = 0; i < N; ++i) {
6         if (i % 2 == 0) {
7             for (int j = 0; j < N / 2; ++j) {
8                 if (2 * j + 1 < N) {
9                     compare_swap(arr[2 * j], arr[2 * j + 1]);
10                }
11            }
12        } else {
13            for (int j = 0; j < (N - 1) / 2; ++j) {
14                compare_swap(arr[2 * j + 1], arr[2 * j + 2]);
15            }
16        }
17    }
18 }
```


Benchmarks



GitHub link:



Thank You!